Securing ML and AI Workloads and Model Endpoints

Slide 01: Title

Welcome to Domain 4, Part 3, the final video for Domain 4 in this theory-only series for the AWS Machine Learning Engineer Associate exam. We have monitored our models and optimized their cost. Now we secure them. In this video we cover least-privilege access, protecting data and models, securing the CI/CD pipeline, network isolation and auditing, authenticating to foundation models, generative AI risks like prompt injection, Amazon Bedrock Guardrails, and how to troubleshoot access problems. Let's begin.

Slide 02: Least privilege starts with IAM

Security on AWS starts with AWS Identity and Access Management, or IAM, and the principle of least privilege. Identity policies control what users and roles can do. Keep actions specific, avoid wildcard resources where you can, and separate development from production. Execution roles are the roles that SageMaker AI assumes when it runs a training job, processing job, or endpoint. The role needs permission to read training data from Amazon S3, pull images from Amazon ECR, and write outputs. Give each workload its own role rather than one powerful shared role. Resource controls add another layer. S3 bucket policies and AWS KMS key policies control who can use data and encryption keys. At the organization level, service control policies set limits for whole accounts, and condition keys can require things like running jobs inside a VPC. Exam tip: when a SageMaker AI job cannot read from S3, check the execution role first.

Slide 03: Protect data, models, and artifacts

Next, we protect the data, the models, and the artifacts. For data access, give each team its own S3 prefix with matching permissions, use AWS Lake Formation for fine-grained table and column access, and always block public access to buckets. For model access, control who can register, approve, and deploy models in the SageMaker Model Registry. Attribute-based access control, or ABAC, uses tags so a project's members can only touch that project's resources. A good practice is that the person who approves a model is not the person who built it. For encryption, use AWS KMS keys to encrypt data at rest in Amazon S3, on training and endpoint storage volumes, and in model artifacts. Use TLS for traffic in transit, and for distributed training you can turn on inter-container traffic encryption. Exam tip: customer managed KMS keys give you control over key policies and rotation.

Slide 04: Secure the CI/CD pipeline

The CI/CD pipeline we built in Domain 3 should also check for security problems. First, scan code. Amazon Inspector can scan code and its dependencies for known vulnerabilities, and the pipeline

should fail the build when it finds critical issues. Second, scan container images. Amazon ECR supports basic scanning when an image is pushed, and enhanced scanning powered by Amazon Inspector, which keeps rescanning images as new vulnerabilities, called CVEs, are published. Third, handle secrets properly. Never put passwords or API keys directly in code or in a buildspec file. Store them in AWS Secrets Manager or Systems Manager Parameter Store, and let the pipeline read them at run time. Secrets Manager can also rotate secrets automatically. Exam tip: continuous scanning of images in ECR points to enhanced scanning with Amazon Inspector.

Slide 05: Isolate the network and audit everything

We covered private networking in Domain 3, so here is a quick recap from a security angle. On the left is network isolation. Run training jobs and endpoints in private subnets, use security groups to allow only the ports you need, and use VPC interface endpoints so calls to SageMaker AI and Amazon Bedrock never cross the public internet. Network isolation mode blocks all outbound calls from a container. On the right is auditing. AWS CloudTrail records every API call, so you can answer who created this endpoint or who invoked this model. AWS Config continuously checks resource settings against rules, for example that every endpoint uses a KMS key. AWS Trusted Advisor flags risky configurations and missed best practices. A common pattern is to send logs to a separate security account so they cannot be tampered with. Exam tip: who did what points to CloudTrail; is this resource configured correctly points to AWS Config.

Slide 06: Authenticate to foundation models securely

How does an application prove it is allowed to call a foundation model? The best default is IAM credentials. When your application runs with an IAM role, on Lambda, EC2, or a container, AWS provides temporary credentials automatically. There are no keys to store, and IAM policies can limit which models the application can invoke. Amazon Bedrock also supports API keys, which make it easy to call Bedrock from tools that expect a simple key. Short-term keys expire quickly, while long-term keys last longer and carry more risk, so they are best kept for development and exploration, with rotation and limited scope. Finally, when you call models from external providers, store their keys in AWS Secrets Manager. The secrets are encrypted with KMS, can rotate automatically, and are retrieved at run time instead of being written into code. Exam tip: the most secure option is usually an IAM role with temporary credentials.

Slide 07: Generative AI brings new security risks

Generative AI adds risks that traditional ML does not have. The biggest is prompt injection. A user writes something like ignore your previous instructions, trying to override the system prompt. It can also be indirect, where the malicious instruction is hidden inside a document or web page that the model retrieves through RAG. Mitigations include clearly separating instructions from user data, validating inputs, and using a prompt attack filter. The second risk is data leakage, where personal information or the system prompt shows up in a response. Output filtering helps here. The third risk is excessive agency. An agent with broad permissions can do real damage if it is

tricked. So give each tool a least-privilege role, limit the tools an agent can use, and require user confirmation before risky actions like deleting data or sending payments.

Slide 08: Amazon Bedrock Guardrails enforce safe outputs

Amazon Bedrock Guardrails apply safety rules to both prompts and responses, and one guardrail can be reused across many models and applications. Content filters block harmful categories, such as hate, insults, sexual content, violence, and misconduct, at a strength you choose. They also include a prompt attack filter that helps detect prompt injection. Denied topics keep an application focused. For example, a banking assistant can refuse to give investment advice. Word filters block specific words and profanity. Sensitive information filters detect personal information like names, email addresses, and social security numbers, and either block the response or mask the data. You can add your own regular expressions too. Contextual grounding checks compare a response against the source documents and the question, and flag answers that are not grounded or not relevant. This reduces hallucinations in RAG applications. Exam tip: mask PII in responses points directly to sensitive information filters in Guardrails.

Slide 09: Troubleshoot access and network issues

When something fails with access denied or a timeout, work through the layers in order. First, read the error carefully. It usually names the principal, the action, and the resource. Second, trace the request in AWS CloudTrail and find the access denied event, which shows exactly which call failed. Third, check every policy that applies: the identity policy, any resource policy like an S3 bucket policy, service control policies, permissions boundaries, and the KMS key policy if the data is encrypted. Remember that an explicit deny anywhere always wins. Fourth, if the error is a timeout rather than access denied, it is usually the network. Check security groups, route tables, and whether the VPC endpoint you need exists. Fifth, verify the fix with the IAM policy simulator, then retry. Exam tip: a job in a private subnet that cannot reach S3 usually needs an S3 gateway endpoint.

Slide 10: A security checklist

Let's finish Domain 4 with this checklist. One: give every user, job, and agent tool least-privilege access through IAM. Two: encrypt data and models at rest with AWS KMS and in transit with TLS. Three: scan code and container images in the CI/CD pipeline, and keep secrets out of code. Four: use private networking, and audit everything with CloudTrail and AWS Config. Five: prefer IAM roles for model access, and store any other keys in Secrets Manager. Six: defend against prompt injection and harmful output with Amazon Bedrock Guardrails. That completes Domain 4 and the full set of exam domains in this series. We have gone from data preparation, to model development, to deployment, and now to operating, monitoring, and securing ML and AI solutions. Thanks for watching, and good luck on the exam.